Convergence and Accuracy of PINNs vs. Classical Finite Difference Schemes for BVPs

Evan Strasdin, V00907185

Department of Mathematics and Statistics

April 8, 2026

Outline

1. Motivation
2. Prior Work
3. Test Problems
4. Finite Difference Methods
5. Nonlinear Solvers (Problem 3)
6. Physics-Informed Neural Networks
7. Results
8. Conclusions

Motivation

- I bet a neural network can do this!

Motivation

- I bet a neural network can do this!
- But: Is it appropriate? (Is it *accurate*? Is it *efficient*?)

Motivation

- I bet a neural network can do this!
- But: Is it appropriate? (Is it *accurate*? Is it *efficient*?)
- Does it beat the competition? (Classical numerical methods have been around for centuries and are very good at solving BVP's.)

Motivation

- I bet a neural network can do this!
- But: Is it appropriate? (Is it *accurate*? Is it *efficient*?)
- Does it beat the competition? (Classical numerical methods have been around for centuries and are very good at solving BVP's.)

Research question: *How do PINNs compare to classical finite difference methods in terms of convergence and accuracy for solving boundary value problems?*

Prior Work

- Raissi et al. (2019): PINN's work

Prior Work

- Raissi et al. (2019): PINN's work
  - Demonstrated efficacy of NN's on forward problems (solution) and inverse problems (unknown parameters) for PDE's.

Prior Work

- Raissi et al. (2019): PINN's work
  - Demonstrated efficacy of NN's on forward problems (solution) and inverse problems (unknown parameters) for PDE's.
- Wang et al. (2022): BUT: PINN's fail to train beyond a certain accuracy threshold

Prior Work

- Raissi et al. (2019): PINN's work
  - Demonstrated efficacy of NN's on forward problems (solution) and inverse problems (unknown parameters) for PDE's.
- Wang et al. (2022): BUT: PINN's fail to train beyond a certain accuracy threshold
  - Gradient descent stalls on the non-convex PDE loss

Prior Work

- Raissi et al. (2019): PINN's work
  - Demonstrated efficacy of NN's on forward problems (solution) and inverse problems (unknown parameters) for PDE's.
- Wang et al. (2022): BUT: PINN's fail to train beyond a certain accuracy threshold
  - Gradient descent stalls on the non-convex PDE loss
  - Changing one weight affects output everywhere, gradients can conflict

Prior Work

- Raissi et al. (2019): PINN's work
  - Demonstrated efficacy of NN's on forward problems (solution) and inverse problems (unknown parameters) for PDE's.
- Wang et al. (2022): BUT: PINN's fail to train beyond a certain accuracy threshold
  - Gradient descent stalls on the non-convex PDE loss
  - Changing one weight affects output everywhere, gradients can conflict

Note: Perdikaris co-authored both!

Prior Work

- Raissi et al. (2019): PINN's work
  - Demonstrated efficacy of NN's on forward problems (solution) and inverse problems (unknown parameters) for PDE's.
- Wang et al. (2022): BUT: PINN's fail to train beyond a certain accuracy threshold
  - Gradient descent stalls on the non-convex PDE loss
  - Changing one weight affects output everywhere, gradients can conflict

Note: Perdikaris co-authored both!

Another note: **Spectral Bias**, Rahaman et al., (2019)

Test Problems

Test Problems

Problem 1 (Poisson, constant coefficient, linear):

$$u'' = -\pi^2 \sin(\pi x), \quad u(0) = 0,\ u(1) = 0$$

Exact: $u(x) = \sin(\pi x)$

Test Problems

Problem 1 (Poisson, constant coefficient, linear):

$$u'' = -\pi^2 \sin(\pi x), \quad u(0) = 0,\ u(1) = 0$$

$$\text{Exact: } u(x) = \sin(\pi x)$$

Problem 2 (variable coefficient, linear):

$$(1 + x^2)\, u'' + 2x\, u' = d(x), \quad u(0) = 1,\ u(1) = 2 + \sin(1)$$

$$\text{Exact: } u(x) = x^2 + \sin(x) + 1$$

Test Problems

Problem 1 (Poisson, constant coefficient, linear):

$$u'' = -\pi^2 \sin(\pi x), \quad u(0) = 0,\ u(1) = 0$$

Exact: $u(x) = \sin(\pi x)$

Problem 2 (variable coefficient, linear):

$$(1 + x^2)\, u'' + 2x\, u' = d(x), \quad u(0) = 1,\ u(1) = 2 + \sin(1)$$

Exact: $u(x) = x^2 + \sin(x) + 1$

Problem 3 (nonlinear):

$$u'' = 3u + x^2 + 10u^3, \quad u(0) = 0,\ u(1) = 0.1$$

Reference solution: Newton shooting at tol 10^{-12}

FDM: Implementation

- Discretize $[0, 1]$ on n interior grid points, spacing $h = 1/(n + 1)$

FDM: Implementation

- Discretize $[0, 1]$ on n interior grid points, spacing $h = 1/(n+1)$
- Prob 1 & 3: centered difference $u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$

FDM: Implementation

- Discretize $[0, 1]$ on n interior grid points, spacing $h = 1/(n+1)$
- Prob 1 & 3: centered difference $u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$
- Prob 2: symmetric differencing for $(a(x)u')'$

FDM: Implementation

- Discretize $[0, 1]$ on n interior grid points, spacing $h = 1/(n+1)$
- Prob 1 & 3: centered difference $u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$
- Prob 2: symmetric differencing for $(a(x)u')'$
- Yields tridiagonal system $A_h \mathbf{U} = \mathbf{F}$, solved in $O(n)$

FDM: Implementation

- Discretize $[0, 1]$ on n interior grid points, spacing $h = 1/(n+1)$
- Prob 1 & 3: centered difference $u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$
- Prob 2: symmetric differencing for $(a(x)u')'$
- Yields tridiagonal system $A_h \mathbf{U} = \mathbf{F}$, solved in $O(n)$
  - ($O(kn)$, k iteration for problem 3)

FDM: Implementation

- Discretize $[0, 1]$ on n interior grid points, spacing $h = 1/(n+1)$
- Prob 1 & 3: centered difference $u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$
- Prob 2: symmetric differencing for $(a(x)u')'$
- Yields tridiagonal system $A_h \mathbf{U} = \mathbf{F}$, solved in $O(n)$
  - ($O(kn)$, k iteration for problem 3)
- Convergence rate $r = 2$, error $\div 4$ when n doubles

FDM: Implementation

- Discretize $[0, 1]$ on n interior grid points, spacing $h = 1/(n+1)$
- Prob 1 & 3: centered difference $u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$
- Prob 2: symmetric differencing for $(a(x)u')'$
- Yields tridiagonal system $A_h \mathbf{U} = \mathbf{F}$, solved in $O(n)$
  - ($O(kn)$, k iteration for problem 3)
- Convergence rate $r = 2$, error $\div 4$ when n doubles
  - $r = \frac{\log(e_n/e_{2n})}{\log(2)}$

FDM: Implementation

- Discretize $[0, 1]$ on n interior grid points, spacing $h = 1/(n+1)$
- Prob 1 & 3: centered difference $u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$
- Prob 2: symmetric differencing for $(a(x)u')'$
- Yields tridiagonal system $A_h \mathbf{U} = \mathbf{F}$, solved in $O(n)$
  - ($O(kn)$, k iteration for problem 3)
- Convergence rate $r = 2$, error $\div 4$ when n doubles
  - $r = \frac{\log(e_n/e_{2n})}{\log(2)}$
  - Error $\approx C h^2$. Double $n \Rightarrow h$ halves $\Rightarrow C\,(h/2)^2 = C\,h^2/4$

FDM: Implementation

- Discretize $[0, 1]$ on n interior grid points, spacing $h = 1/(n+1)$
- Prob 1 & 3: centered difference $u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$
- Prob 2: symmetric differencing for $(a(x)u')'$
- Yields tridiagonal system $A_h \mathbf{U} = \mathbf{F}$, solved in $O(n)$
  - ($O(kn)$, k iteration for problem 3)
- Convergence rate $r = 2$, error $\div 4$ when n doubles
  - $r = \frac{\log(e_n/e_{2n})}{\log(2)}$
  - Error $\approx C\,h^2$. Double $n \Rightarrow h$ halves $\Rightarrow C\,(h/2)^2 = C\,h^2/4$
- Condition number: Predictable accuracy ceiling

Nonlinear Solvers (Problem 3)

- No closed-form solution, so we need iterative solvers

Nonlinear Solvers (Problem 3)

- No closed-form solution, so we need iterative solvers
- **Shooting methods** (reduce BVP to IVP):

Nonlinear Solvers (Problem 3)

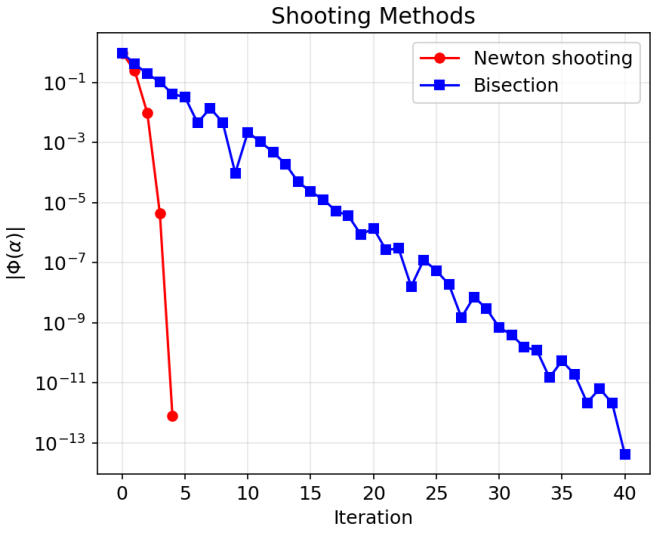
- No closed-form solution, so we need iterative solvers
- **Shooting methods** (reduce BVP to IVP):
  - Newton shooting: quadratic convergence, 5 iterations

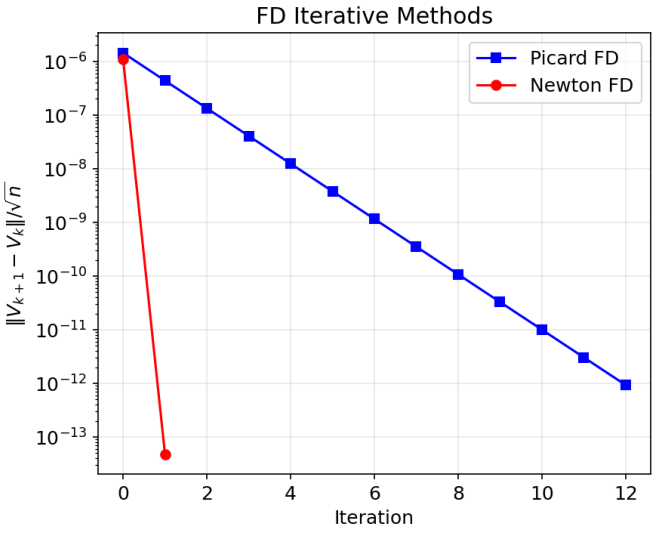
Nonlinear Solvers (Problem 3)

- No closed-form solution, so we need iterative solvers
- **Shooting methods** (reduce BVP to IVP):
  - Newton shooting: quadratic convergence, 5 iterations
  - Bisection: linear convergence, 41 iterations

Nonlinear Solvers (Problem 3)

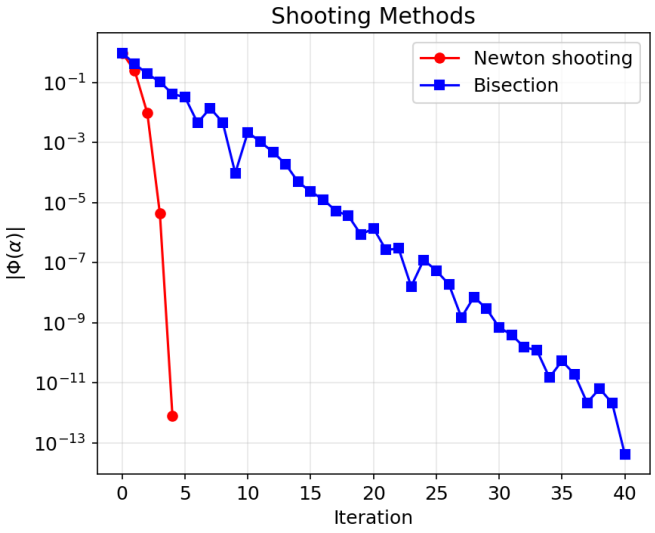
- No closed-form solution, so we need iterative solvers
- **Shooting methods** (reduce BVP to IVP):
  - Newton shooting: quadratic convergence, 5 iterations
  - Bisection: linear convergence, 41 iterations
- **Finite difference methods** (solve discrete system directly):

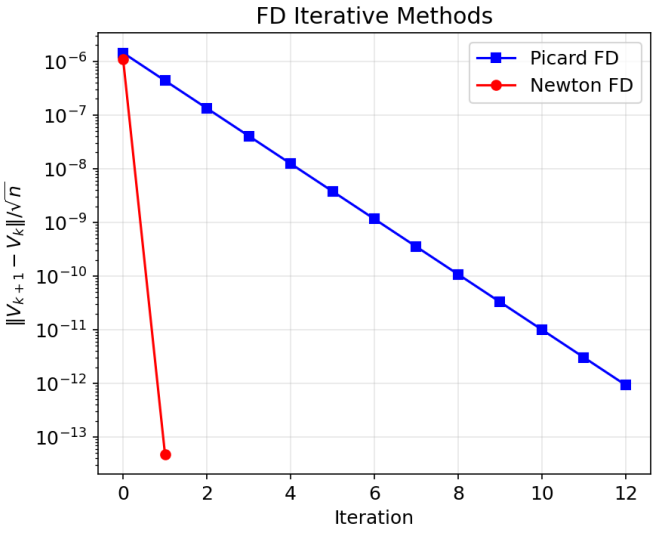




Nonlinear Solvers (Problem 3)

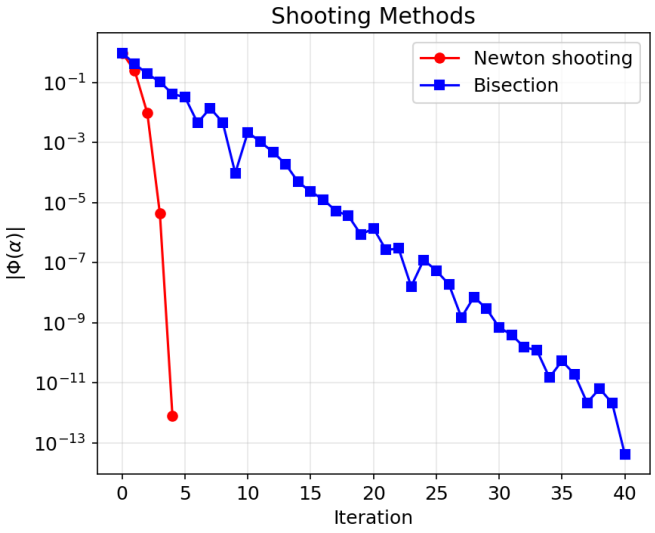
- No closed-form solution, so we need iterative solvers
- **Shooting methods** (reduce BVP to IVP):
  - Newton shooting: quadratic convergence, 5 iterations
  - Bisection: linear convergence, 41 iterations
- **Finite difference methods** (solve discrete system directly):
  - Picard FD: linear convergence, 13 iterations

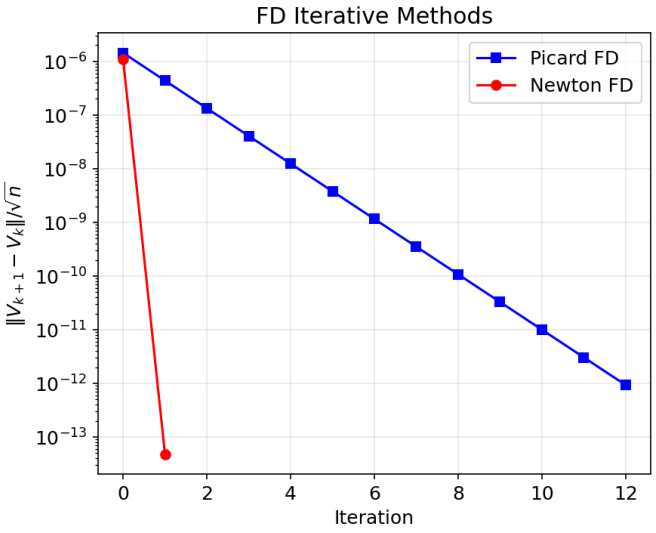




Nonlinear Solvers (Problem 3)

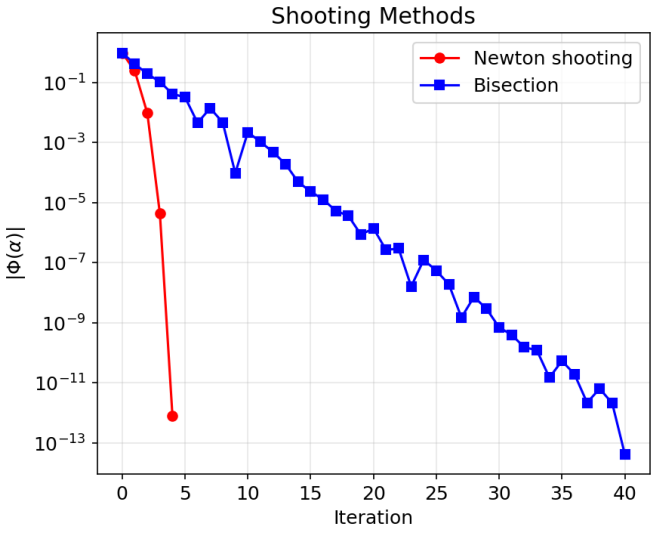
- No closed-form solution, so we need iterative solvers
- **Shooting methods** (reduce BVP to IVP):
  - Newton shooting: quadratic convergence, 5 iterations
  - Bisection: linear convergence, 41 iterations
- **Finite difference methods** (solve discrete system directly):
  - Picard FD: linear convergence, 13 iterations
  - Newton FD: quadratic convergence, **2 iterations**

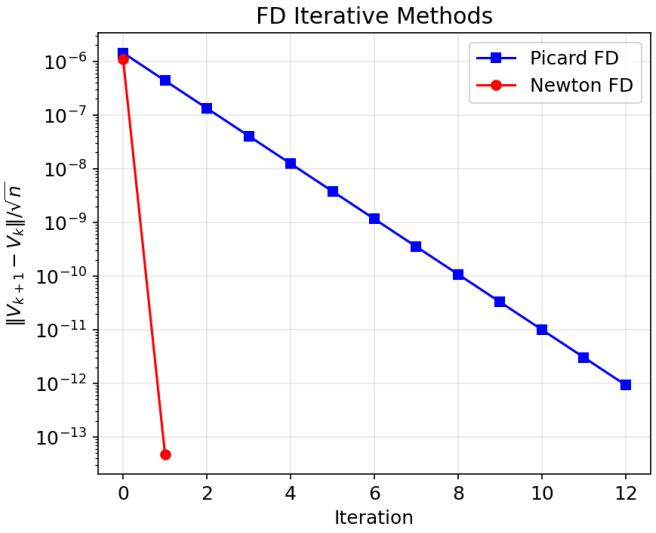




Nonlinear Solvers (Problem 3)

- No closed-form solution, so we need iterative solvers
- **Shooting methods** (reduce BVP to IVP):
  - Newton shooting: quadratic convergence, 5 iterations
  - Bisection: linear convergence, 41 iterations
- **Finite difference methods** (solve discrete system directly):
  - Picard FD: linear convergence, 13 iterations
  - Newton FD: quadratic convergence, **2 iterations**
- Reference solution: Newton shooting at tol 10^{-12}





PINNs: Formulation

- Neural network as trial function, hard BC:

$$\hat{u}(x;\theta) = g_0(1-x) + g_1 x + x(1-x)\,\mathrm{NN}(x;\theta)$$

PINNs: Formulation

- Neural network as trial function, hard BC:

$$\hat{u}(x;\theta) = g_0(1-x) + g_1 x + x(1-x)\,\mathrm{NN}(x;\theta)$$

- Loss $=$ mean squared PDE residual at N_c collocation points:

$$\mathcal{L}(\theta) = \frac{1}{N_c}\sum_{i=1}^{N_c} \left|r_i\right|^2$$

PINNs: Formulation

- Neural network as trial function, hard BC:

$$\hat{u}(x;\theta) = g_0(1-x) + g_1 x + x(1-x)\,\mathrm{NN}(x;\theta)$$

- Loss $=$ mean squared PDE residual at N_c collocation points:

$$\mathcal{L}(\theta) = \frac{1}{N_c}\sum_{i=1}^{N_c} |r_i|^2$$

- Derivatives via automatic differentiation (exact, no discretization error)

PINNs: Formulation

- Neural network as trial function, hard BC:

$$\hat{u}(x;\theta) = g_0(1-x) + g_1 x + x(1-x)\,\mathrm{NN}(x;\theta)$$

- Loss $=$ mean squared PDE residual at N_c collocation points:

$$\mathcal{L}(\theta) = \frac{1}{N_c}\sum_{i=1}^{N_c} |r_i|^2$$

- Derivatives via automatic differentiation (exact, no discretization error)
- Two-phase training: Adam (10k epochs) $\rightarrow$ L-BFGS (refinement)

PINNs: Formulation

- Neural network as trial function, hard BC:

$$\hat{u}(x;\theta) = g_0(1-x) + g_1 x + x(1-x)\,\mathrm{NN}(x;\theta)$$

- Loss $=$ mean squared PDE residual at N_c collocation points:

$$\mathcal{L}(\theta) = \frac{1}{N_c}\sum_{i=1}^{N_c} |r_i|^2$$

- Derivatives via automatic differentiation (exact, no discretization error)
- Two-phase training: Adam (10k epochs) $\rightarrow$ L-BFGS (refinement)
- Architecture: 3 hidden layers $\times$ 50 neurons, $\tanh$, $\approx$ 3,200 parameters

PINNs as Nonlinear Collocation

- Classical: write $\hat{u} = \sum c_k \phi_k(x)$ (polynomial/spline basis), solve a linear system for c_k

PINNs as Nonlinear Collocation

- Classical: write $\hat{u} = \sum c_k \phi_k(x)$ (polynomial/spline basis), solve a linear system for c_k
- PINN: replace the basis with a neural network, minimize residual via gradient descent

PINNs as Nonlinear Collocation

- Classical: write $\hat{u} = \sum c_k \phi_k(x)$ (polynomial/spline basis), solve a linear system for c_k
- PINN: replace the basis with a neural network, minimize residual via gradient descent
- Key difference from FDM: N_c (resolution) and parameter count (DOF) are decoupled

PINNs as Nonlinear Collocation

- Classical: write $\hat{u} = \sum c_k \phi_k(x)$ (polynomial/spline basis), solve a linear system for c_k
- PINN: replace the basis with a neural network, minimize residual via gradient descent
- Key difference from FDM: N_c (resolution) and parameter count (DOF) are decoupled
- Known limitations:

PINNs as Nonlinear Collocation

- Classical: write $\hat{u} = \sum c_k \phi_k(x)$ (polynomial/spline basis), solve a linear system for c_k
- PINN: replace the basis with a neural network, minimize residual via gradient descent
- Key difference from FDM: N_c (resolution) and parameter count (DOF) are decoupled
- Known limitations:
  - **Spectral bias**: networks learn low frequencies first (Rahaman et al., 2019)

PINNs as Nonlinear Collocation

- Classical: write $\hat{u} = \sum c_k \phi_k(x)$ (polynomial/spline basis), solve a linear system for c_k
- PINN: replace the basis with a neural network, minimize residual via gradient descent
- Key difference from FDM: N_c (resolution) and parameter count (DOF) are decoupled
- Known limitations:
  - **Spectral bias**: networks learn low frequencies first (Rahaman et al., 2019)
  - **Non-convex loss**: gradients from different collocation points can conflict (Wang et al., 2022)

PINNs as Nonlinear Collocation

- Classical: write $\hat{u} = \sum c_k \phi_k(x)$ (polynomial/spline basis), solve a linear system for c_k
- PINN: replace the basis with a neural network, minimize residual via gradient descent
- Key difference from FDM: N_c (resolution) and parameter count (DOF) are decoupled
- Known limitations:
  - **Spectral bias**: networks learn low frequencies first (Rahaman et al., 2019)
  - **Non-convex loss**: gradients from different collocation points can conflict (Wang et al., 2022)
- Increasing N_c does not help: the bottleneck is the optimizer, not the sampling

Results: FDM Convergence

	Problem 1		**Problem 2**		**Problem 3**	
n	Error	Rate	Error	Rate	Error	Rate
7	1.26e-2	–	3.91e-3	–	3.75e-4	–
31	7.79e-4	1.92	2.45e-4	1.91	2.39e-5	1.91
127	4.86e-5	1.96	1.53e-5	1.98	1.49e-6	1.98
511	3.07e-6	1.93	9.59e-7	1.99	9.33e-8	1.99

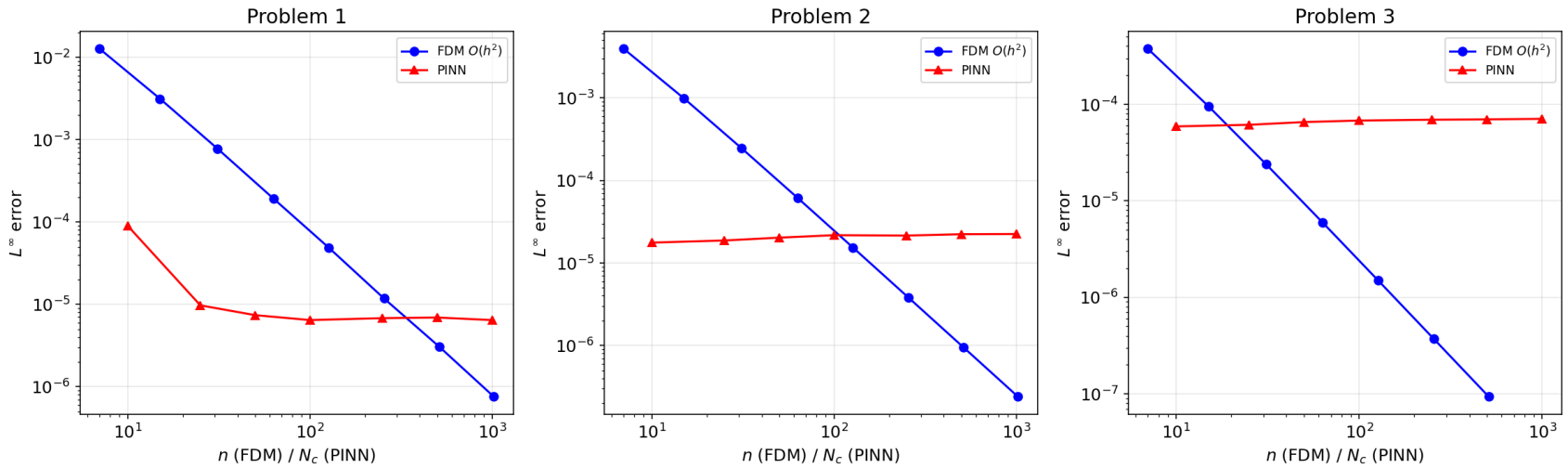
- All three problems: rate $\to 2$ as predicted by BVP convergence theorem

Results: FDM Convergence

	Problem 1		**Problem 2**		**Problem 3**	
n	Error	Rate	Error	Rate	Error	Rate
7	1.26e-2	–	3.91e-3	–	3.75e-4	–
31	7.79e-4	1.92	2.45e-4	1.91	2.39e-5	1.91
127	4.86e-5	1.96	1.53e-5	1.98	1.49e-6	1.98
511	3.07e-6	1.93	9.59e-7	1.99	9.33e-8	1.99

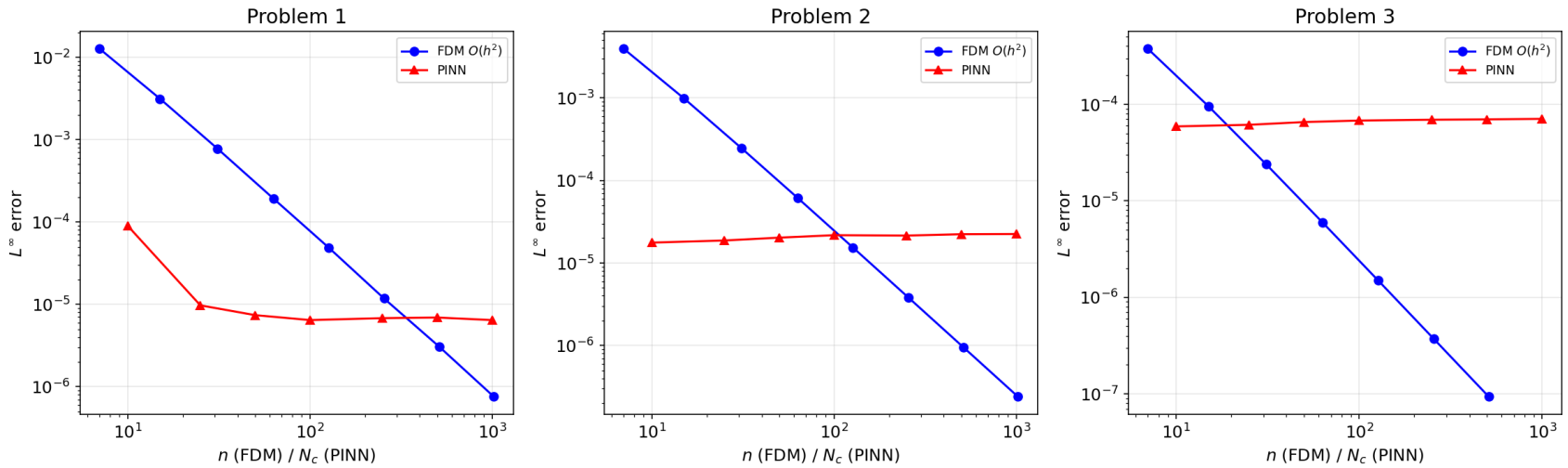
- All three problems: rate $\to 2$ as predicted by BVP convergence theorem
- Problem 3 converges identically despite nonlinearity

Results: Accuracy vs. Sampling Density



- FDM (blue): consistent slope -2.

Results: Accuracy vs. Sampling Density



- FDM (blue): consistent slope -2.
- PINN (red): **flat**. Increasing N_c does not reduce error.

Results: PINN Plateau

N_c	**Problem 1**	**Problem 2**	**Problem 3**
10	8.92e-5	1.76e-5	5.90e-5
50	7.39e-6	2.03e-5	6.55e-5
100	6.44e-6	2.17e-5	6.79e-5
500	6.91e-6	2.23e-5	6.98e-5
1000	6.44e-6	2.24e-5	7.07e-5

- Errors stagnate at $\sim 10^{-5}$ to 10^{-6} regardless of N_c

Results: PINN Plateau

N_c	**Problem 1**	**Problem 2**	**Problem 3**
10	8.92e-5	1.76e-5	5.90e-5
50	7.39e-6	2.03e-5	6.55e-5
100	6.44e-6	2.17e-5	6.79e-5
500	6.91e-6	2.23e-5	6.98e-5
1000	6.44e-6	2.24e-5	7.07e-5

- Errors stagnate at $\sim 10^{-5}$ to 10^{-6} regardless of N_c
- Convergence rate ≈ 0 (vs. FDM rate $= 2$)

Results: PINN Plateau

N_c	Problem 1	Problem 2	Problem 3
10	8.92e-5	1.76e-5	5.90e-5
50	7.39e-6	2.03e-5	6.55e-5
100	6.44e-6	2.17e-5	6.79e-5
500	6.91e-6	2.23e-5	6.98e-5
1000	6.44e-6	2.24e-5	7.07e-5

- Errors stagnate at $\sim 10^{-5}$ to 10^{-6} regardless of N_c
- Convergence rate ≈ 0 (vs. FDM rate $= 2$)
- Consistent with Wang et al. (2022): spectral bias + non-convex loss

Results: PINN Plateau

N_c	Problem 1	Problem 2	Problem 3
10	8.92e-5	1.76e-5	5.90e-5
50	7.39e-6	2.03e-5	6.55e-5
100	6.44e-6	2.17e-5	6.79e-5
500	6.91e-6	2.23e-5	6.98e-5
1000	6.44e-6	2.24e-5	7.07e-5

- Errors stagnate at $\sim 10^{-5}$ to 10^{-6} regardless of N_c
- Convergence rate ≈ 0 (vs. FDM rate $= 2$)
- Consistent with Wang et al. (2022): spectral bias + non-convex loss
- Compare: FDM at $n = 511$ achieves 9.33e-8 (Problem 3)

Results: Cost

Wall-Clock Time

Method	Time
FDM ($n = 255$)	$\sim 200\,\mu$s
FDM ($n = 1023$)	$\sim 370\,\mu$s
PINN training	41–70 s
PINN inference	$\sim 400\,\mu$s

- PINN training: $10^5\times$ slower than FDM

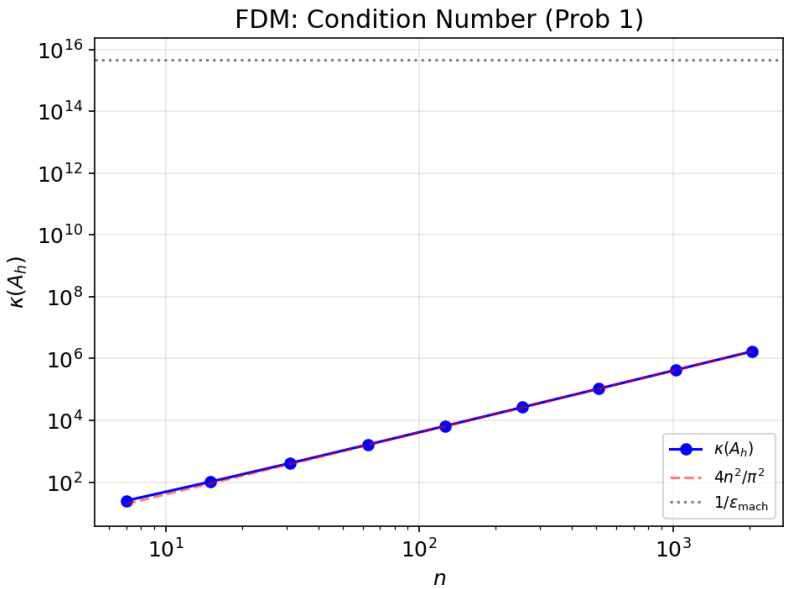
Results: Cost

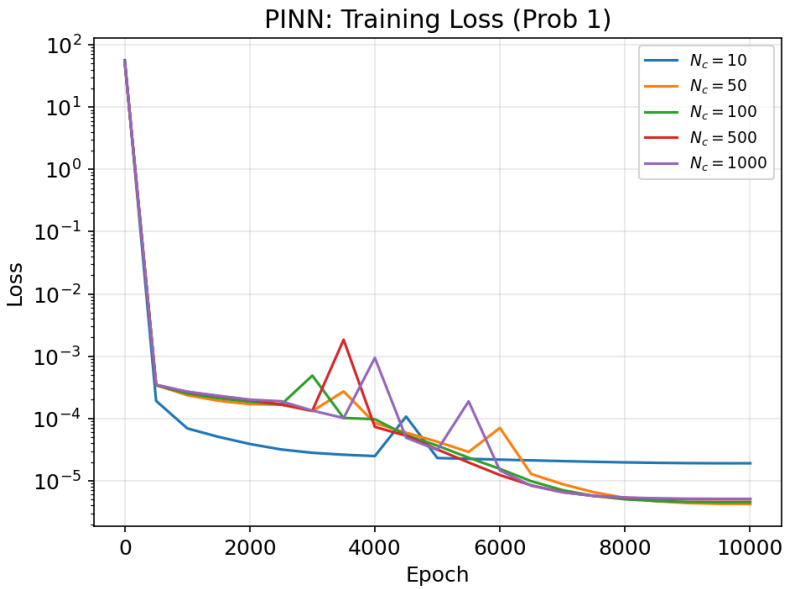
Wall-Clock Time

Method	Time
FDM ($n = 255$)	$\sim 200\,\mu$s
FDM ($n = 1023$)	$\sim 370\,\mu$s
PINN training	41–70 s
PINN inference	$\sim 400\,\mu$s

- PINN training: $10^5\times$ slower than FDM
- PINN inference (post-training) $\approx$ FDM solve time

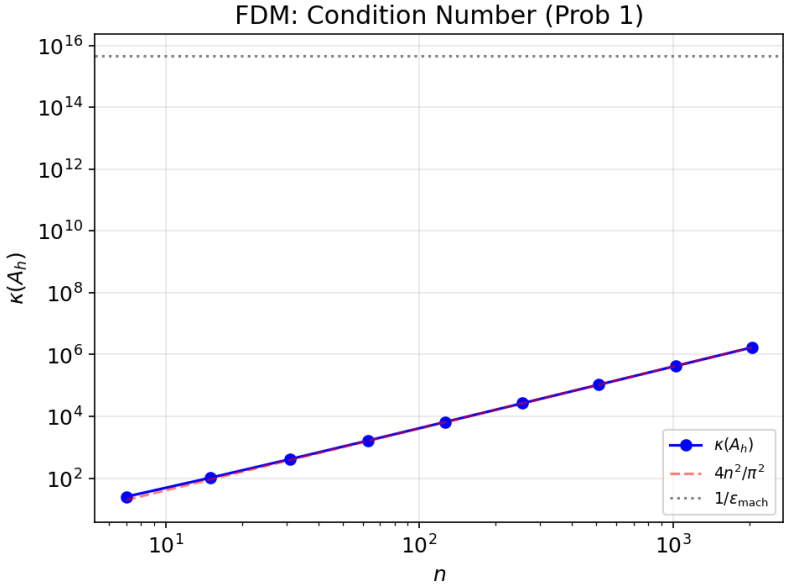
Results: Conditioning vs. Loss Landscape

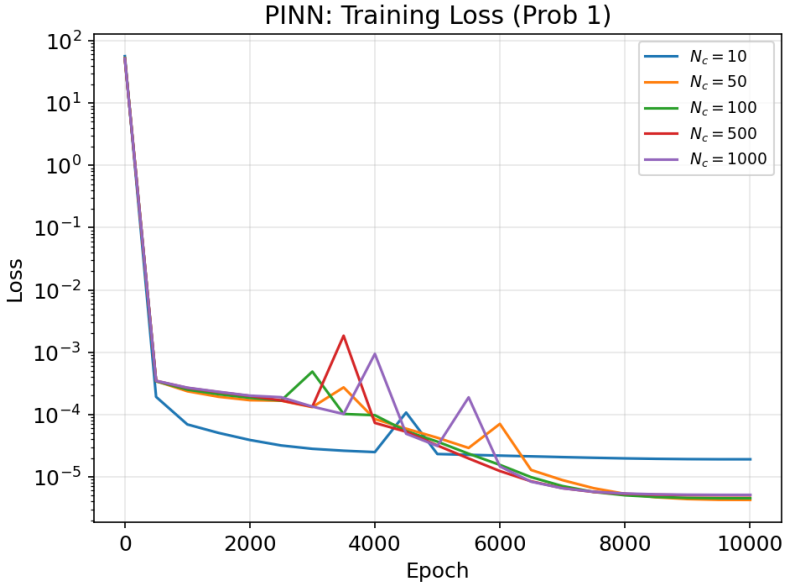




- FDM ceiling: predictable ($\kappa \cdot \epsilon_{\text{mach}}$)

Results: Conditioning vs. Loss Landscape





- FDM ceiling: predictable ($\kappa \cdot \epsilon_{\text{mach}}$)
- PINN ceiling: empirical, no theory to predict it

Conclusions

	FDM	PINN
Convergence rate	$O(h^2)$ (proven)	No guarantee
Best accuracy	$\sim 10^{-7}$ (limited by κ)	$\sim 10^{-5}$ (plateau)
Wall-clock time	μs	13–78 s (training)
Mesh required?	Yes	No
Nonlinear problems	Iterative solvers	Same framework
Inverse problems	No mechanism	Natural fit

- FDM is superior for well-posed 1D BVPs
- PINN accuracy plateau consistent with Wang et al. (2022)
- PINNs may shine in higher dimensions, complex geometries, inverse problems... further research?

References I

M. Raissi, P. Perdikaris, G. E. Karniadakis.
Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear PDEs.
J. Comput. Phys., 378:686–707, 2019.

S. Wang, X. Yu, P. Perdikaris.
When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective.
J. Comput. Phys., 449:110768, 2022.

N. Rahaman et al.
On the Spectral Bias of Neural Networks.
ICML 2019, pp. 5301–5310, 2019.

References II

B. Khouider.
MATH 449/549 Chapter 4: Boundary Value Problems (Lecture Slides).
University of Victoria, Spring 2026.

B. Khouider.
MATH 449/549 Lecture Notes.
University of Victoria, Spring 2026.